5. Run the application, and click the RadioButton controls to switch between view states. You should see that the two controls appear and disappear as you click the RadioButton controls to switch view states.

Declaring view states in MXML

A view state is represented by the <s:State> tag set and is always assigned a name:

```
<s:State name="myNewState">
... state declaration ...
</s:State>
```

To declare one or more view states in a main application or custom component file, wrap the <s:State> declaration tags within <s:states> tags:

```
<?xml version="1.0" encoding="utf-8"?>
<s:Application xmlns:fx="http://ns.adobe.com/mxml/2009"
xmlns:s="library://ns.adobe.com/flex/spark"
xmlns:mx="library://ns.adobe.com/flex/mx">
<s:states>
<s:State name="State1"/>
<s:State name="oneway"/>
</s:states>
... declare visual objects ...
</s:Application>
```

The <s:states> element must always be declared as a child element of the Application or component root; you cannot nest state declarations within other child MXML tag sets unless they are in an <fx:Component> tag set (used with custom item renderers.

Technically speaking, it doesn't matter whether the <s:states> tag set is at the top, bottom, or middle of the MXML code, as long as it's a direct child of the MXML file's root element and doesn't separate visual components from each other. Once a state has been declared, you can then indicate which controls are a part of the state with the includeIn or excludeFrom attributes. You can also modify properties, styles, and event handlers by adding new MXML attributes to the affected component declarations.

Managing view states in components

You can declare a view state inside a custom component. The rules are the same as for a main application file: You can only apply the view state to the entire component, not to its nested child objects. You can then control that